Sorting application using a 6 DOF robotic arm



Sort Arm-6 DOF



جامعة الإسكندرية
ALEXANDRIA UNIVERSITY

Course: Robotics

Course Code: **EEP 433**

Instructor: Mohammed El-Habrouk

Team: 5

Team Members

Name	ID
Abd El-Rhman Muhammad Saad	**19017058**
Mariem Mamdouh Ibrahim	**21011335**
Rowan Tamer Ismail	**21010537**
Nada Mahmoud Youssef	**21011026**
Marina Said Botros	**21011026**
Asmaa Galal Ahmed	**19015429**
Razan Ramzy Mohammed	**21010515**
Tasneem Alaa El-Shandidi	**21010396**

1. Objectives

Abstract

This lab booklet documents the design, implementation, testing, and troubleshooting of SortArm-6DOF, a six-degree-of-freedom robotic manipulator integrated into a conveyor sorting cell. The system detects colored boxes using an IR sensor and neglects the white ones, stops the conveyor, executes a pick-and-place routine with the robotic arm and gripper, and then resumes conveyor motion. White boxes are ignored.

Primary objectives

- Detect colored boxes reliably and trigger a deterministic pick-and-place sequence.
- Integrate Arduino Uno control with six servos, conveyor motors, and sensors.
- Demonstrate a repeatable sorting routine suitable for small-scale manufacturing tasks.
- Document hardware, software, test logs, and lessons learned across the term.

2. System Description and Workflow

2.1. System Overview

The sorting cell comprises a conveyor, an IR detection station, the SortArm-6DOF manipulator with a servo-driven gripper, a motor driver for the conveyor, and an Arduino Uno controller. The system is powered by an external 5 V, 20 A supply to ensure stable servo operation.

2.2. Functional Workflow

1. **Conveyor runs** continuously transferring boxes.
2. **IR sensor** monitors objects (Black boxes) at the detection point.
3. **Black box detected**: controller stops conveyor and initiates pick sequence.
4. **Arm moves** from home pose to pick pose.
5. **Gripper closes** to secure the object.
6. **Arm moves** to place the pose and releases the object.
7. **Arm returns** to home pose and conveyor restarts.

2.3. Key Functional Requirements

- **Reliable detection** of colored boxes using IR sensor thresholding.
- **Deterministic servo motion** with sufficient current capacity.
- **Safe conveyor control** with clean start/stop transitions.
- **Mechanical clearance** and cable management to avoid collisions.

2.4. Hardware Components

Component	Specification
Controller	Arduino UNO
Power Supply	5V, 20A external
Servos	Servos
Conveyor Motors	Dual-shaft DC motors
Motor Driver	**L293D** motor driver IC
Conveyor Material	MDF, 4.8 mm thickness
Detection Sensor	IR reflective sensor
Gripper	Two-finger mechanical gripper

2.5. Suggested Pin Mapping to Arduino UNO

- IR sensor: digital pin 2
- Buzzer: digital pin 7
- Servos: servo_1: pin 3, servo_2: pin 5, servo_3: pin 6, servo_4: pin 9, servo_5: pin 10, servo_6: pin 11
- NPN base pin to control 2 DC motors: Digital Pin 7

3. Mechanical Design and Visual Documentation

3.1. Robotic Arm

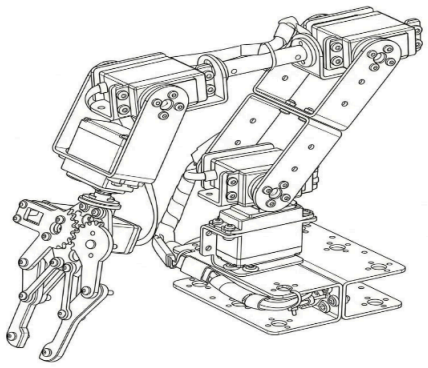


Figure 1 Robotic arm design

3.2. Robotic Arm Analysis

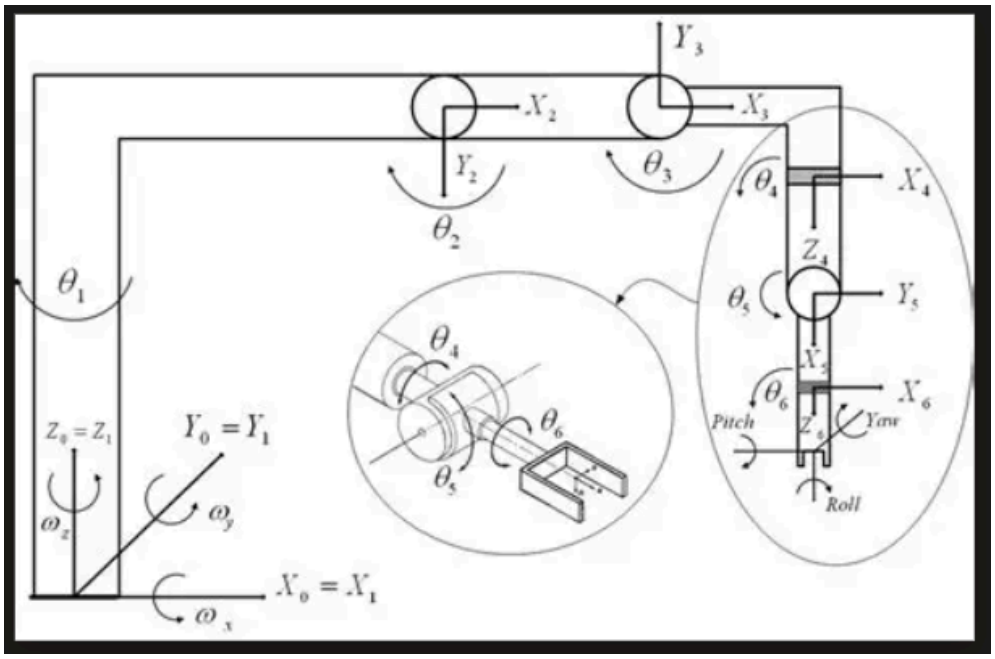
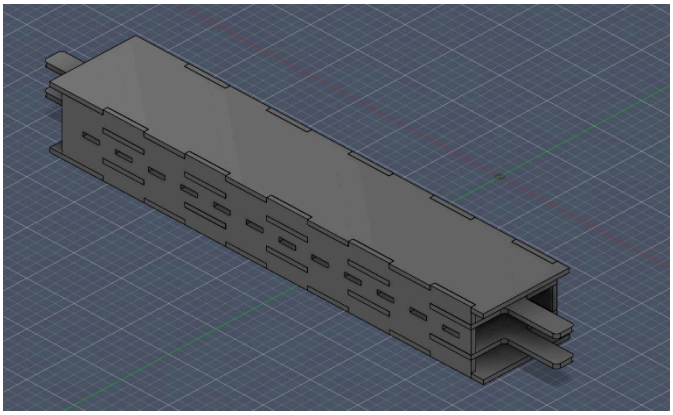


Figure 2 Robotic arm analysis

3.3. Conveyor Design

Dimensions: 54.24 x 9.11 cm



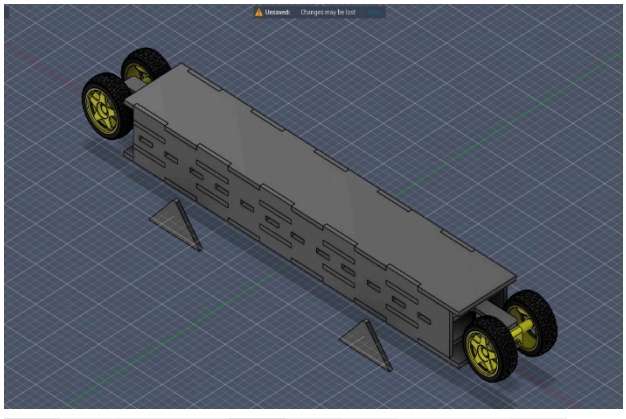


Figure 4: conveyor designing phase

3.4. Assembled System

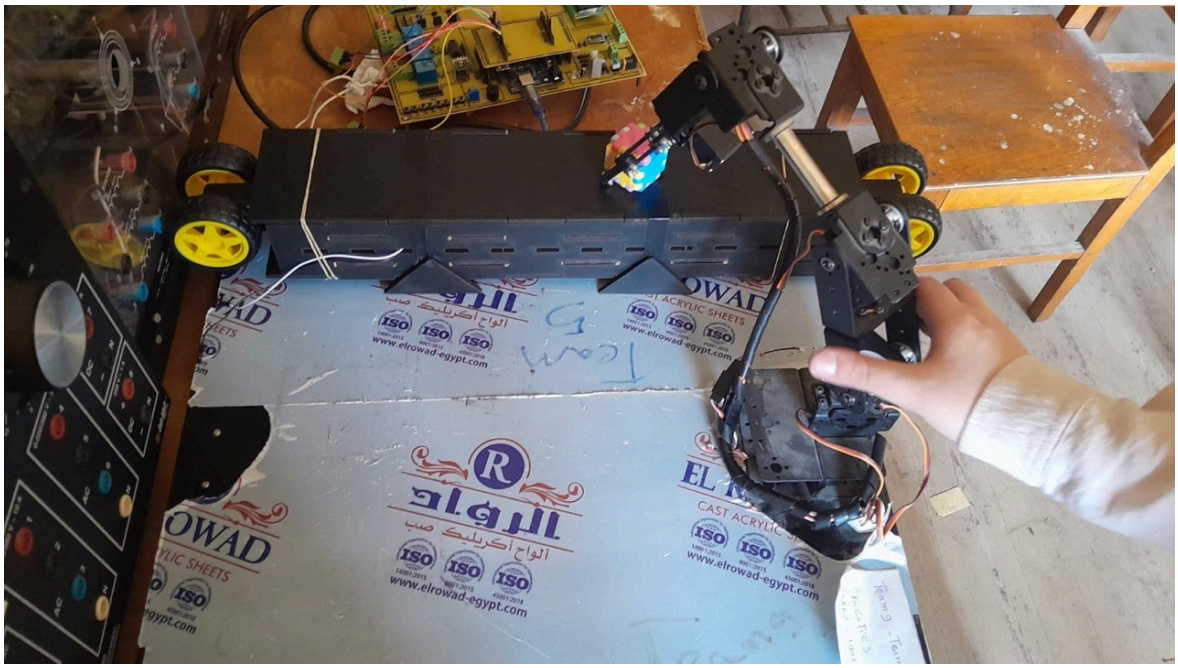


Figure 5 Assembled system, including both the arm and the conveyor

4. Electric Wiring and Circuit Schematic

4.1. Hardware Connection

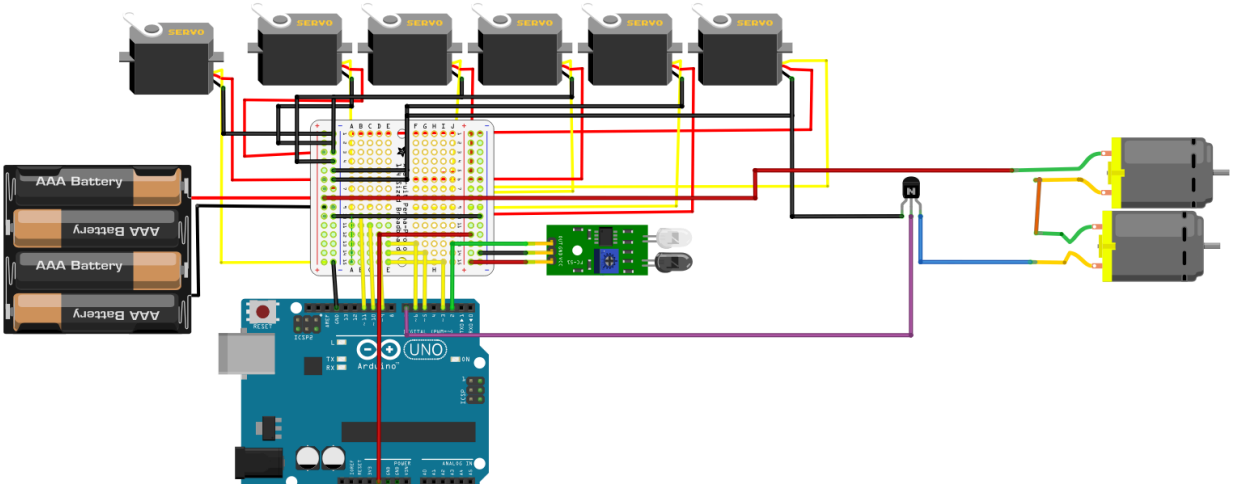


Figure 6: Hardware Components Connections

4.2. Circuit Schematic

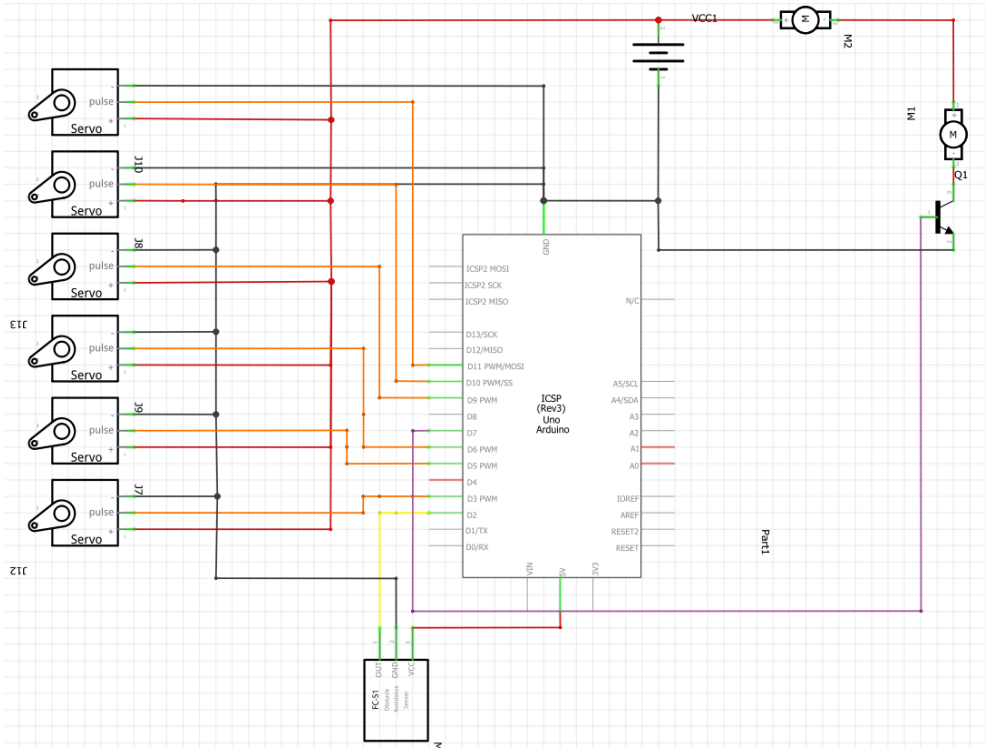


Figure 7 Connection Schematic

4.3. Notes

- The system uses a single external 5 V, 20 A supply for servos and motors. Arduino logic is powered from the same supply with a common ground.

- The NPN Transistor controls the two DC conveyor motors with the Microcontroller digital signal based IR readings .
- Servo signal lines are routed to Arduino PWM-capable pins.
- The IR reflective sensor provides a digital detection signal to Arduino pin 2. The detection logic is implemented.

5. Software and Code

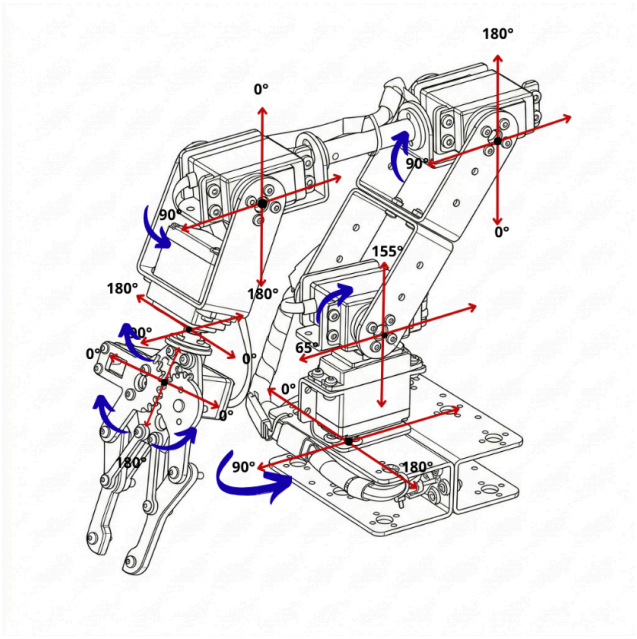
5.1. Development Environment

Arduino IDE, C++ using the Servo library.

5.2. Design Approach

5.2.1. Testing and Calibration

Testing and calibration were conducted to detect the limits angles of each servo in addition to rotation direction.



Arm Part	<90	Middle	>90	Max Angle	Min Angle
S1	Left	90	Right	180	0
S2	Down	65	Up	140	65
S3	Down	90	Up	180	0
S4	Down	90	Up	180	0
S5	Right	90	Left	180	0
S6	OPEN	145	CLOSE	110	170

5.2.2. Calibration Sketch

```
#include <Servo.h>

// Create servo objects for each joint
Servo servoA;  // S1
Servo servoB;  // S2
Servo servoC;  // S3
Servo servoD;  // S4
Servo servoE;  // S5
Servo servoF;  // S6

// Store current angle values
int angleA = 90;
int angleB = 90;
int angleC = 90;
int angleD = 90;
int angleE = 90;
int angleF = 90;

// Input buffer
String inputString = "";
bool stringComplete = false;

void setup() {
  Serial.begin(9600);
  Serial.println("Enter commands like: A 90 B 120 C 60 D 50");

  // Attach servos to pins
  servoA.attach(3);   // S1
  servoB.attach(5);   // S2
  servoC.attach(6);   // S3
  servoD.attach(9);   // S4
  servoE.attach(10);  // S5
  servoF.attach(11);  // S6

  // Move all servos to initial angle smoothly
  moveSmooth(servoA, angleA);
  moveSmooth(servoB, angleB);
  moveSmooth(servoC, angleC);
```

```
  moveSmooth(servoD, angleD);
  moveSmooth(servoE, angleE);
  moveSmooth(servoF, angleF);
  inputString.reserve(50);
}

void loop() {
  if (stringComplete) {
    processCommand(inputString);
    inputString = "";
    stringComplete = false;
  }
}

void serialEvent() {
  while (Serial.available()) {
    char inChar = (char)Serial.read();
    if (inChar == '\n' || inChar == '\r') {
      stringComplete = true;
    } else {
      inputString += inChar;
    }
  }
}

void processCommand(String command) {
  command.trim();
  command += " ";
  int index = 0;

  while (index < command.length()) {
    char servoChar = toupper(command.charAt(index));
    index++;

    while (index < command.length() && command.charAt(index) == ' ') index++;

    String angleStr = "";
    while (index < command.length() && isDigit(command.charAt(index))) {
      angleStr += command.charAt(index);
      index++;
```

```
    }

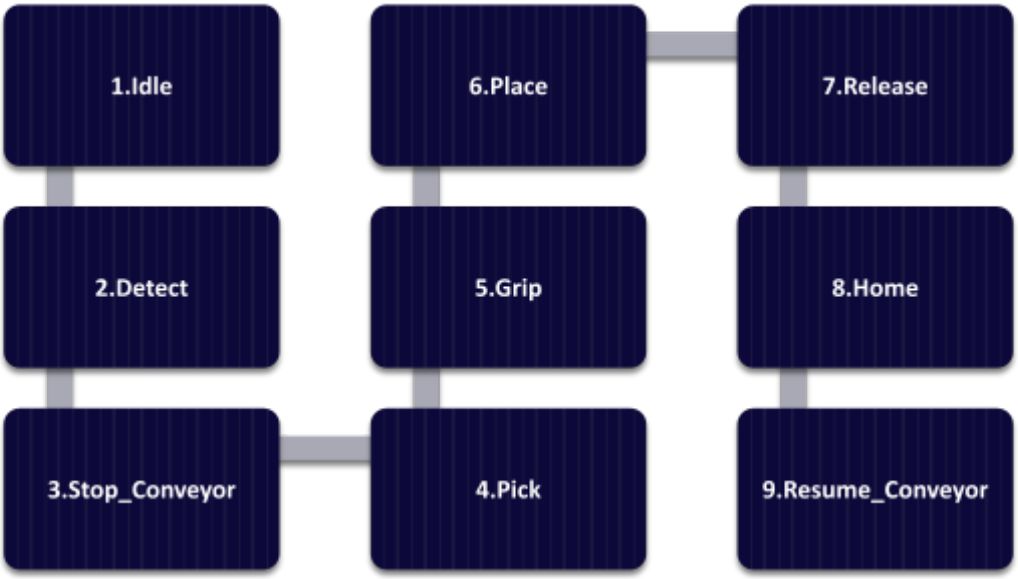
    int angle = angleStr.toInt();
    angle = constrain(angle, 0, 180);

    switch (servoChar) {
      case 'A':
        moveSmooth(servoA, angle);
        angleA = angle;
        Serial.print("A set to "); Serial.println(angle);
        break;
      case 'B':
        moveSmooth(servoB, angle);
        angleB = angle;
        Serial.print("B set to "); Serial.println(angle);
        break;
       case 'C':
         moveSmooth(servoC, angle);
         angleC = angle;
         Serial.print("C set to "); Serial.println(angle);
         break;
       case 'D':
         moveSmooth(servoD, angle);
         angleD = angle;
         Serial.print("D set to "); Serial.println(angle);
         break;
       case 'E':
         moveSmooth(servoE, angle);
         angleE = angle;
         Serial.print("E set to "); Serial.println(angle);
         break;
       case 'F':
         moveSmooth(servoF, angle);
         angleF = angle;
         Serial.print("F set to "); Serial.println(angle);
         break;
       default:
         Serial.print("Invalid servo name: ");
         Serial.println(servoChar);
     }
```

```
124.
125.       while (index < command.length() && command.charAt(index) == ' ') index++;
126.     }
127.   }
128.
129.   // Smooth servo movement
130.   void moveSmooth(Servo servo, int targetAngle) {
131.     int currentAngle = servo.read();
132.     if (currentAngle < targetAngle) {
133.       for (int i = currentAngle; i <= targetAngle; i++) {
134.         servo.write(i);
135.         delay(10);
136.       }
137.     } else {
138.       for (int i = currentAngle; i >= targetAngle; i--) {
139.         servo.write(i);
140.         delay(10);
141.       }
142.     }
143.   }
```

5.3. Operational Sketch

- Fixed pose angles used initially; calibrated angles were obtained by the used calibration code.



```
-  #include "Servo.h"
-
-  // Define the IR sensor and buzzer pins
-  const int irPin = 2;
-  const int Conveyor = 7;

-  Servo servo_1, servo_2, servo_3, servo_4, servo_5, servo_6;
-
-  void setup() {
-    Serial.begin(9600);
-
-    pinMode(irPin, INPUT);
-    pinMode(Conveyor, OUTPUT);
-
-    servo_1.attach(3);
-    servo_2.attach(5);
-    servo_3.attach(6);
-    servo_4.attach(9);
-    servo_5.attach(10);
-    servo_6.attach(11);
-    Serial.println("System Initialized. Monitoring IR Sensor...");
-    //initial Positions
-    servo_1.write(90);
-    servo_2.write(105);
-    servo_3.write(120);
-    servo_4.write(30);
-    servo_5.write(90);
-    servo_6.write(120);
-  }
-
-  void loop() {
-    int irVal = digitalRead(irPin);
-    Serial.println(irVal);
-    // Buzzer Indication
-    digitalWrite(buzzer, !irVal);
-    // --- Sequence Logic ---
-    if (irVal == LOW) {
-      Serial.println("IR State: [ BLACK BOX DETECTED ] - Starting
   Sequence");
-
```

```
-      digitalWrite(Conveyor, LOW);
-      pickPosition_1(2000);
-      pickPosition_2(2000);
-      gripPosition(2000);
-      releasePosition(2000);
-      ungripPosition(2000);
-      homePosition(2000);
-      Serial.println("IR State: [ SEQUENCE FINISHED ] - Waiting...");
-    } else {
-      Serial.print("IR Sensor Value: ");
-      Serial.println("IR State: [ NO BLACK BOX DETECTED ] -
   Waiting........ ");
-      digitalWrite(Conveyor, HIGH);
-      homePosition(200);
-    }
-  }
-
-  // --- Functions with delay_time Argument ---
-
-  void homePosition(int delay_time) {
-    servo_1.write(90);
-    servo_2.write(105);
-    servo_3.write(120);
-    servo_4.write(30);
-    servo_5.write(90);
-    servo_6.write(120);
-    delay(delay_time);
-  }
-
-  void pickPosition_1(int delay_time) {//take a position
-    servo_1.write(80);
-    servo_2.write(140);
-    servo_3.write(110);
-    servo_4.write(80);
-    servo_5.write(160);
-    servo_6.write(110);
-    delay(delay_time);
-  }
-
-  void pickPosition_2(int delay_time) {// go nearer only s2 moves
```

```
-    servo_1.write(80);
-    servo_2.write(110);
-    servo_3.write(110);
-    servo_4.write(80);
-    servo_5.write(160);
-    servo_6.write(110);
-    delay(delay_time);
-  }
-  void gripPosition(int delay_time) {// griP action only s6 moves
-    servo_1.write(80);
-    servo_2.write(110);
-    servo_3.write(110);
-    servo_4.write(80);
-    servo_5.write(160);
-    servo_6.write(150);
-    delay(delay_time);
-  }
-
-  void releasePosition(int delay_time) {// same as grib but only s1
   moves
-    servo_1.write(150);
-    servo_2.write(110);
-    servo_3.write(110);
-    servo_4.write(80);
-    servo_5.write(160);
-    servo_6.write(150);
-    delay(delay_time);
-  }
-
-  void ungripPosition(int delay_time) {// same as release but only s6
   moves to ungriP
-    servo_1.write(150);
-    servo_2.write(110);
-    servo_3.write(110);
-    servo_4.write(80);
-    servo_5.write(160);
-    servo_6.write(110);
-    delay(delay_time);
-  }
```

5.4. Fixed Poses

Position	S1 A	S2 B	S3 C	S4 D	S5 E	S6 F
Home	90	105	120	30	90	120
Pick_1	80	140	110	80	160	110
pick_2	80	110	110	80	160	110
Grip	80	110	110	80	160	150
release	150	110	110	80	160	150
ungrip	150	110	110	80	160	110

6. ROS Integration

6.1. I. Structural Assembly and Spatial Synthesis

- **6DOF Integration:** Unified over 30 SolidWorks (**SLDPRT**) components, including high-torque servos and structural brackets.
- **Structural Assembly:** Secured rigid body integrity using a hierarchy of **Fastened Mates**.
- **Kinematic Mapping:** Implemented **Revolute Mates** to define the six rotational degrees of freedom (DoF) required for robotic motion.
- Software used: onshape website



6.2. Joint Calibration and Initial State Definition (In Progress)

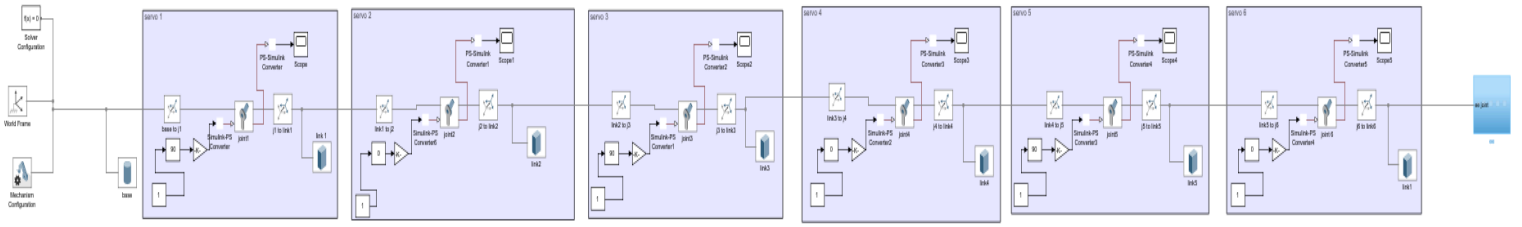
- **Zero-Position Configuration:** Defined the "Home" state to establish a global reference frame for forward kinematics.
- **Angular Constraints:** Applied joint limits (0° to 180°) to mirror physical servo hardware and prevent simulation self-collisions.

6.3. URDF Translation and Morphological Export (Yet to be)

- **URDF Conversion:** Translated the SolidWorks assembly (**SLDASM**) into a **Unified Robotics Description Format** via a programmatic exporter.
- **Automated Data Extraction:** Extracted **Inertial Tensors** and **Link Offsets** directly from 3D geometry to ensure high-fidelity simulation.
- **ROS 2 Optimization:** Enabled accurate **Inverse Kinematics (IK)** and path planning within the **MoveIt 2** framework.

7. MATLAB Simulation

Design and simulate a 3D 6-Degree of Freedom (6-DOF) serial robotic manipulator using MATLAB/Simulink and Simscape Multibody.
The robot consists of a fixed base, 6 moving links, and 6 revolute joints connected in a serial kinematic chain.



7.2. Methodology & Simulink Implementation

Global Environment Setup:

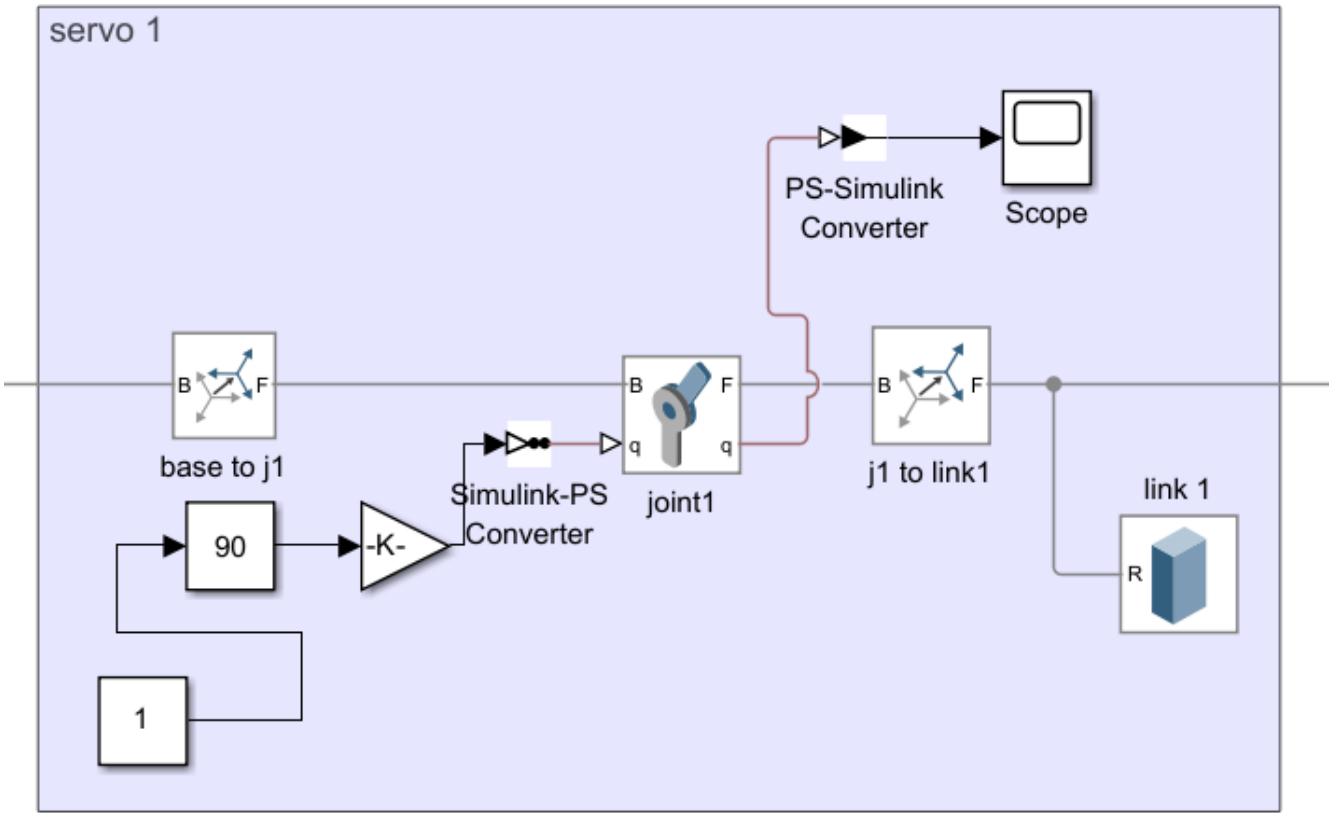
- Solver Configuration: Defines the numerical solver parameters for the physical network.
- World Frame: Provides the absolute, immobile coordinate reference system (0,0,0).
- Mechanism Configuration: Sets the gravity vector ([0, 0, -9.81]) and uniform mechanical tolerances.

Modular Subsystem Design: The model uses a highly structured, cascaded subsystem architecture (joint 1 to joint 6). This modular design keeps the top-level model clean and allows for individual troubleshooting of each joint.
Component Breakdown: The standard blocks used inside each block:

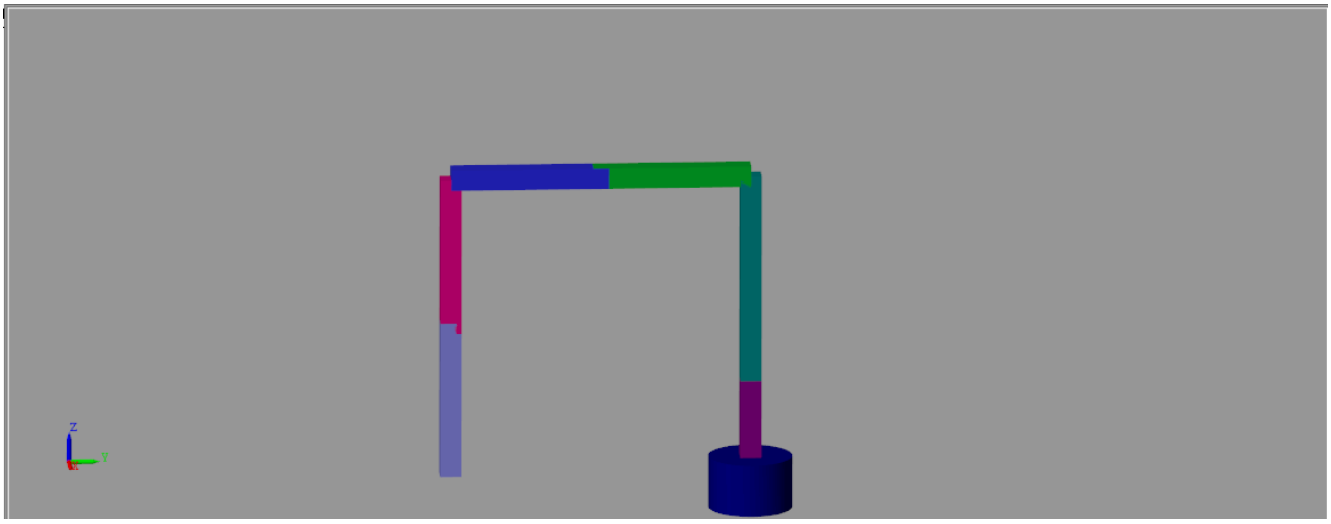
- Rigid Transforms: Used to define the link lengths (translations) and the relative angular offsets (rotations) between adjacent joint axes.
- Revolute Joints: Act as the physical degrees of freedom, configured to rotate around their local Z-axes.

Signal Conversion: Simulink-PS Converter: Converts the mathematical input command (angles/degrees) into a physical signal that the Simscape joint can interpret.

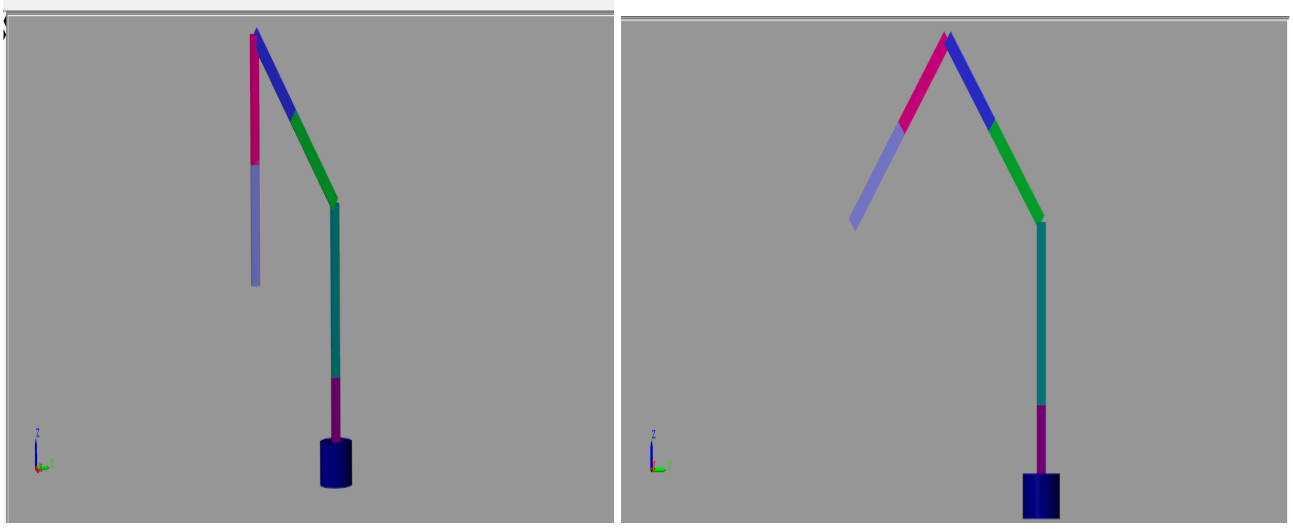
- **PS-Simulink Converter:** Captures the physical joint state feedback (q) and transforms it back into a standard Simulink signal for data logging and viewing on the **Scope**



Simulation Results: According to angles given for each joint the simulation takes its shape



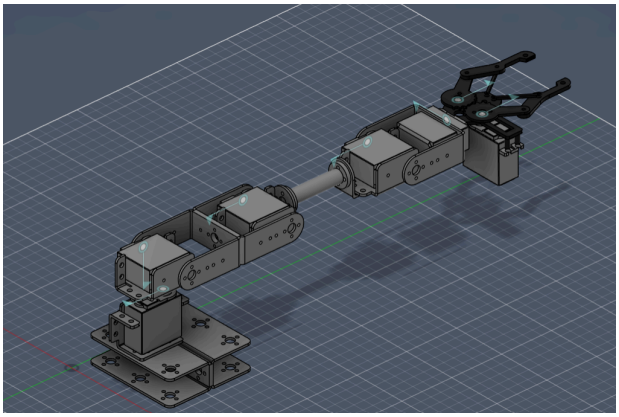
Home position



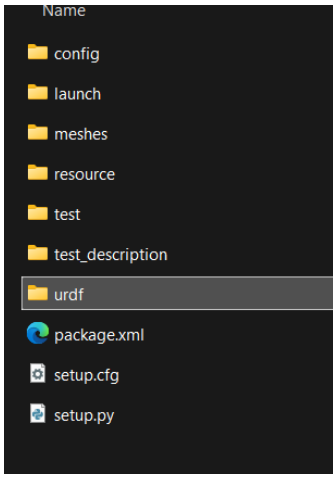
Pick position

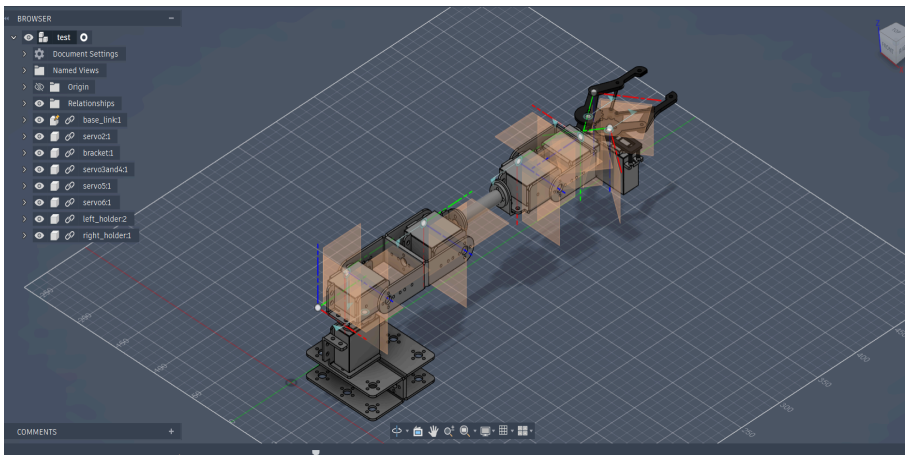
9.Simulation in ROS2 RVIZ

- Exporting the URDF File



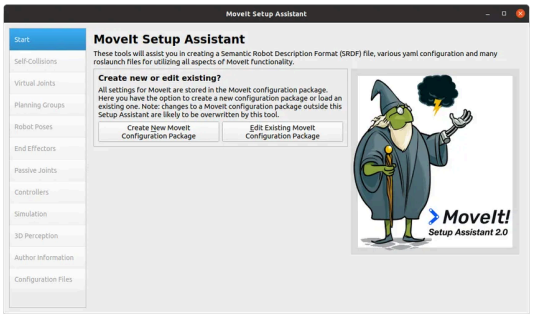
We used Autodesk Fusion 360 to combine the STEP files of the robot parts and organize them into links (the rigid parts) and joints (the moving parts). We made sure all links were correctly connected and that the rotation axes were set to the Z-axis. Finally, we used the Fusion 360 URDF exporter plugin to generate the .urdf and .xacro configuration files inside our ROS 2 package.





- MoveIt Setup Assistant

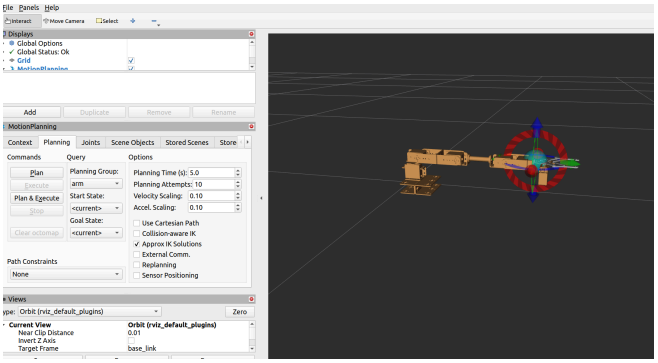
  Next, we loaded the generated XACRO file into the moveit_setup_assistant. After setting up the required parameters—such as planning groups and self-collision matrices—the assistant generated a custom configuration package. This package contains the launch files needed to run the simulation.

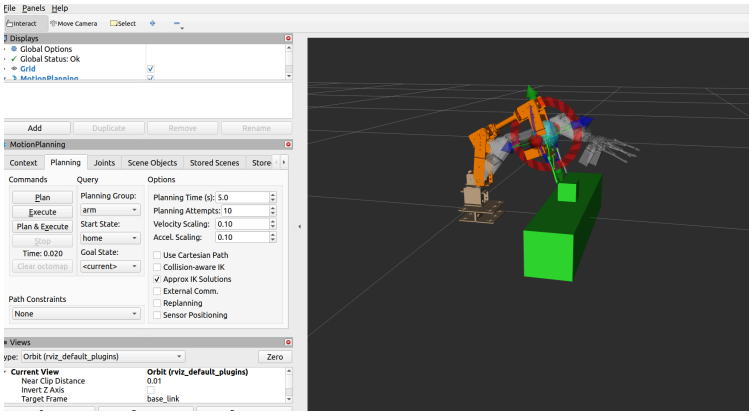


- RViz Simulation and Testing

  We launched the simulation in RViz using the generated demo.launch.py script. To test a realistic pick-and-place scenario, we added a conveyor belt and a box to the scene.

  We then tested the motion planning:





  - Clicking Plan uses MoveIt's Inverse Kinematics (IK) algorithm to calculate a safe path from the start position to the goal position.
  - Clicking Execute sends this path to the joint controllers, generating the exact servo angles needed to move the physical hardware.

8. Testing, Results and Troubleshooting

- **Test plan and procedures**

  - Power and wiring verification: confirm common ground and measure servo rail under load.
  - Servo checks: sweep each servo individually and record mechanical limits.
  - IR calibration: log digitalRead values for black and white boxes at the detection point.
  - Conveyor control: test start/stop with NPN Transistor.
  - Dry runs: execute pick/place without objects to verify collision-free motion.
  - Live runs: run with objects at low throughput and record success rate.

- **Observations and Results Summary**

  - servo_6 (gripper) exhibited intermittent erratic behavior during some runs. Diagnostics performed included swapping servos, checking wiring, and verifying supply stability. The issue is under continued investigation.
  - Multiple servos got replaced due to overheating thus failure during the testing phase.
  - Conveyor prototypes were evaluated; the printed conveyor with the robotic arm demonstrated acceptable alignment and repeatability for the pick-and-place routine.

- **Troubleshooting Summary**

- Problem 1: **The gripper servo cuts out unexpectedly or overheats until it burns out.**

- What We Did:

- Separated the power and signal wires.
- Tested the servo by itself.
- Eased up on the mounts/fasteners to reduce mechanical friction.
- Checked the rotation limits to find the proper operating range: narrow range 70°

- Problem 2: **Servos Low Torques make movement angles not reliable.**